Applicability of Non-Asymptotic Optimizations in Range Filters

Andrei Ogurtsov

May 16, 2026

Abstract

We implement and evaluate Approximate Range Emptiness (ARE) filters for LSM-tree storage systems. Starting from the information-theoretically optimal construction of Goswami et al. (SODA 2015), we develop a family of practical filters that explore different hash function designs: TODO

Contents

Chapter 1

Introduction

1.1 Motivation

1.2 Existing Range Filters

1.3 Notation

General. We identify L-bit keys with the integers $\{0, 1, \ldots, 2^L - 1\}$ via the standard binary representation. Lexicographic comparison of bit strings of the same length coincides with numerical comparison of the corresponding integers, so we use the integer view throughout (this is the natural formulation when the construction performs arithmetic such as rotations or reductions modulo a power of two).

Sets and parameters.

L	Key length in bits.
$U = [0, 2^L)$	Universe of L-bit keys.
n	Number of input keys, $n = \lvert S \rvert$.
$S \subseteq U$	The set of keys in a Range Filter, $\lvert S \rvert = n$.
$Y = U \setminus S$	Non-keys (all points not in S).
$\mathcal{L}$	Maximum query range length.
ε	Target false positive rate.

Hash function and fingerprints.

h	Hash function $h \colon U \to U'$.
K	Fingerprint length in bits, $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$.
$U' = [0, 2^K)$	Mapped universe.

$r = 2^K$	Mapped universe size.
$S' = h(S)$	Fingerprints of the input keys.
$Y' = h(Y)$	Images of non-keys under h.

Metrics.

FPR	False Positive Rate.
BPK	Bits Per Key: total filter size divided by n.

Structures.

ERE	Exact Range Emptiness — zero-error range filter.
ARE	Approximate Range Emptiness — range filter with one-sided error $\leq \varepsilon$.

1.4 Problem Statement

Let $U = [0, 2^L)$ be the universe of L-bit keys (Section 1.3). Given a set $S \subseteq U$ of n keys, a maximum query range length $\mathcal{L}$, and a target false positive rate ε, the *approximate range emptiness problem* asks to build a data structure that answers "$[a, b] \cap S = \emptyset$?" for intervals of length $\leq \mathcal{L}$, with:

- **No false negatives:** if $[a, b] \cap S \neq \emptyset$, the answer is always "not empty".
- **Bounded false positives:** if $[a, b] \cap S = \emptyset$, the answer is "not empty" with probability at most ε.

Let $Y = U \setminus S$ denote all keys not in the filter.

1.4.1 Information-Theoretic Bounds

Goswami et al. [2015] establish the following bounds.

Theorem 1 (Lower bound, Goswami et al. [2015, §2])**.** *Any data structure solving the range emptiness problem requires at least* $n\big(\log_2(\mathcal{L}/\varepsilon) - c\big)$ *bits for an absolute constant* $c > 0$.

Remark 1. *The lower bound is worst-case over the input: it holds for every set* $S \subseteq U$ *with* $|S| = n$*, with no assumptions on the structure or distribution of* S*.*

Remark 2. *In practice, real datasets are often more regular. If the input is known to have additional structure, a data structure may use that structure to reduce the required space below the general bound.*

Theorem 2 (Upper bound, Goswami et al. [2015, §3])**.** *The lower bound of Theorem 1 is achieved up to an additive* $O(n)$ *term via two layers:*

1. *A locality-preserving hash (Section 4.1)* $h\colon U \to U'$*, where* $U' = [0, 2^K)$ *and* $r = |U'| = 2^K$*. The hash partitions* U *into blocks of size* r*, applies an independent rotation within each block, and then maps the shifted block onto* U' *by translation. The hash is drawn from a pairwise-independent family, so for any two distinct keys* $x_1, x_2 \in U$*,* $\Pr[h(x_1) = h(x_2)] \leq 1/2^K$*.*

2. *An* ***Exact Range Emptiness (ERE)*** *(Section 2.1) structure over* $S' = h(S) \subseteq U'$*: a succinct structure with zero error and space* $n \log_2(r/n) + O(n)$ *bits, which answers "*$[a', b'] \cap S' = \emptyset$*?".*

The resulting space is $n \log_2(\mathcal{L}/\varepsilon) + O(n)$ bits, or equivalently $\log_2(\mathcal{L}/\varepsilon) + O(1)$ bits per key — matching the lower bound.

The fingerprint length $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ controls accuracy: because $\Pr[h(x_1) = h(x_2)] \leq 1/2^K$ for any two distinct keys, the overall error probability of the construction is bounded by $n\mathcal{L}/2^K$, which we constrain to be $\leq \varepsilon$. Thus, increasing K by one bit halves the false positive rate. After ERE compression (which subtracts $\log_2 n$ for block indexing), the effective cost per key is $K - \log_2 n = \log_2(\mathcal{L}/\varepsilon) + O(1)$.

1.4.2 The Role of the Hash Function

The hash h projects U down to U'. Under this projection, the keys S map to fingerprints $S' = h(S)$, and the non-keys $Y = U \setminus S$ map to $Y' = h(Y)$.

A false positive occurs when Y' overlaps with S': a query point $y \in Y$ has $h(y) \in S'$, so the structure cannot distinguish it from a real key. We call the false pre-images of a fingerprint $x' \in S'$ its **phantom points** — all non-keys $y \in Y$ such that $h(y) = x'$. The fewer phantoms fall within query ranges, the lower the False Positive Rate (FPR).

The ideal hash would map S' and Y' to disjoint regions of U' — zero overlap, zero false positives. In practice this is impossible within the space budget, but the choice of h determines how well S' and Y' separate for a given data distribution. This motivates two directions explored in this work: alternative hash function designs (Chapter 4) and data-adaptive segmentation strategies that bypass hashing entirely on dense regions (Chapter 5).

Chapter 2

Exact Range Emptiness

2.1 Succinct Representation (SODA 2015, §3.2)

The inner layer of every approximate filter is an *Exact Range Emptiness* (ERE) structure: given a universe U' of K-bit keys and a set $S' \subseteq U'$ of n keys, it answers "$[a, b] \cap S' = \emptyset$?" exactly (no false positives or false negatives).

Remark 3. *Throughout this chapter, S' denotes the input to ERE; in the ARE pipeline (Chapter 4), $S' = h(S)$.*

2.1.1 High-Level Structure

To achieve $n \cdot (K - \log_2(n) + O(1)) = n \log_2(r/n) + O(n)$ bits [Goswami et al., 2015], the structure divides the universe U' into $B = 2^{\lfloor \log_2 n \rfloor}$ equal-sized blocks. Conceptually, ERE has two levels: a succinct navigation layer over blocks, and a compact local representation of the keys.

All keys inside one block have a common $\log_2 B = \lfloor \log_2 n \rfloor$ bits prefix. Therefore, key suffixes can be stored in the array with $w = K - \lfloor \log_2 n \rfloor$ bits per key. The succinct navigation layer is used to determine block boundaries.

2.1.2 Original ERE Structure

We now describe the original ERE construction of Goswami et al. [2015] more precisely. It consists of the following parts:

1. D_1 succinct bit vector: stores block occupancy: $D_1[i] = 1$ iff block i contains at least one key.
2. D_2 succinct bit vector: encodes the sizes of non-empty blocks in unary form. If a non-empty block i contains n_i keys, then it contributes a 1 followed by n_i zeros.
3. A_{ds} the array of structures: each data structure consists of:

   The sorted list of suffixes

   A Weak Prefix Search index over those suffixes (Section 2.1.6).

A query range is split into at most three parts: a sequence of complete blocks and up to two partially covered boundary blocks. The complete-block part is answered by the global succinct layer using D_1. The boundary parts are answered by locating the corresponding local structure in A_{ds} via D_1 and D_2, and then checking emptiness inside the block. The local check uses two weak prefix queries (Section 2.1.6) as an $O(1)$ index into the bucket's sorted suffix list; the stored suffixes themselves witness (non-)emptiness via a final comparison. TODO: btw i do not like forward refs, maybe I need to chenge order of sections?

2.1.3 Local Search in Buckets

The original paper [Goswami et al., 2015] suggests a Weak Prefix Search (WPS) [Belazzougui et al., 2010] structure based on a Hollow Z-Fast Trie (ZFT) [Belazzougui et al., 2009] for $O(1)$ worst-case local queries. We use binary search on bit-packed suffixes, with an adaptive fallback to linear scan for small buckets, instead. At the bucket sizes that occur in practice, binary search — or even linear scan on the smallest buckets — is faster than an $O(1)$ ZFT lookup, whose hidden constant factor is large, and additionally avoids the ZFT's metadata overhead. Even the worst-case bucket of k^* keys takes ∼50–55 bytes at $K = 64$ (Table 2.3), fitting in a single cache line, so a linear scan reduces to one sequential memory access. Details are given in Section 2.1.6.

2.1.4 One-Vector Optimization

In the optimized variant, we simplify the navigation layer. Instead of storing occupancy and non-empty block offsets separately in D_1 and D_2, we replace them with a single succinct bit-vector D. Every block i with n_i keys is encoded as a single 1 followed by n_i zeros, whether the block is empty or not. In particular, an empty block (with $n_i = 0$) is encoded by a single 1. One final sentinel 1 is appended at the end.

Effect on space and navigation. Let M denote the number of non-empty blocks. In the original layout, the metadata size is

$$|D_1| + |D_2| = B + (n + M + 1).$$

In the optimized layout, the metadata becomes

$$|D| = B + n + 1.$$

The exact metadata reduction is M bits. Operationally, this means that the old two-stage lookup through D_1 and D_2 is replaced by direct navigation via SELECT on D.

Metadata footprint on uniform input. Table 2.1 reports the metadata BPK on n uniform 64-bit keys for $n \in \{2^{20}, 2^{24}, 2^{28}\}$. The information-theoretic delta is ∼0.63 BPK, matching the analytic prediction $M/n \to 1 - e^{-1} \approx 0.632$ (each block is independently empty with probability e^{-1}, Section 2.1.5). The physical delta is ∼0.80 BPK, amplified by a constant ∼26% overhead from the rank/select index — intrinsic to succinct bit vectors and required for the $O(1)$ navigation guarantees. Both deltas are stable across n.

	Num (logical bits)		**Alloc** (with rank/select index)	
n	two-vector	one-vector	two-vector	one-vector
2^{20}	2.63	2.00	3.33	2.53
2^{24}	2.63	2.00	3.33	2.53
2^{28}	2.63	2.00	3.33	2.53
Δ	-0.63 BPK (-24%)		-0.80 BPK (-24%)	

Table 2.1: Metadata BPK for the two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout, on n uniform 64-bit keys. *Num*: logical bitvector length (D_1.NUM()+D_2.NUM() vs. D.NUM()); matches the analytic $|D_1|+|D_2| = B+(n+M+1)$ and $|D| = B+n+1$ formulas above. *Alloc*: physical succinct-bit-vector footprint (bits plus pointer, rank, and select indices). The suffix array A_{ds} is excluded — identical in both layouts.

Against the $\sim$ 20 BPK total ERE footprint — dominated by the suffix array A_{ds}, which is identical between the two layouts — the absolute saving of 0.80 BPK looks small. But it is a $\sim$ 24% reduction in metadata: the only part that can shrink without information loss. Cross-distribution measurements are deferred to Chapter 6 (Table 6.2).

Effect on query operation count. The single-vector layout does not strictly reduce the number of RANK/SELECT calls in every case: the original two-vector layout could short-circuit empty boundary blocks via a D_1.BIT() probe, while the new layout always pays at least two SELECT(D) per boundary block. However, in the two most common ARE query patterns — short ranges that hit a single non-empty block, or that span two adjacent non-empty blocks — the new layout saves *one* and *three* RANK/SELECT calls per query, respectively, with long ranges that scan both boundary buckets saving up to four. The structural advantage is that every RANK/SELECT now hits one contiguous succinct bit vector D instead of alternating between D_1 and D_2, so the auxiliary index pages and cache lines loaded by the first call are likely to serve subsequent calls in the same query. The full per-case breakdown is given in Table 2.2; end-to-end query latency on uniform and clustered data is reported in Chapter 6 (Table 6.3).

Query type	(D_1, D_2) ops	D ops	Δ	Note
Same block, non-empty	$1\,R + 2\,S = 3$	$2\,S = 2$	$\mathbf{-1}$	**Most frequent, hot path**
Same block, empty	0	$2\,S = 2$	$+2$	Cold-path regression
Adjacent, both non-empty	$2\,R + 4\,S = 6$	$3\,S = 3$	$\mathbf{-3}$	**Most frequent, hot path**
Adjacent, both empty	0	$3\,S = 3$	$+3$	Cold-path regression
Long range, both boundaries non-empty	$4\,R + 4\,S = 8$	$4\,S = 4$	-4	Op count halved
Long range, intermediate non-empty	$2\,R = 2$	$4\,S = 4$	$+2$	More ops, single vector
Long range, all empty	$2\,R = 2$	$4\,S = 4$	$+2$	Cold-path regression

Table 2.2: Number of RANK/SELECT operations per query case for the original two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout (Section 2.1.4). R denotes RANK on D_1; S denotes SELECT on D_2 or D as appropriate. Bolded rows are the dominant query patterns in typical ARE workloads. BIT probes and bucket-scan operations are not counted (identical between layouts).

TODO: write about better mem layout

Relation to Elias-Fano coding and Grafite. The single bitvector D is equivalent to Elias-Fano coding [Elias, 1974, Fano, 1971]: D is exactly the unary-encoded high-bits bitvector, and the packed suffix array A_{ds} plays the role of the low-bits array. We derived this layout by first rejecting WPS: Section 2.1.6 shows that adaptive linear or binary search over packed suffixes is 8–160× faster (Table 2.5), making the two-vector (D_1, D_2) layout — which exists to support the WPS access pattern — redundant. Grafite [Costa et al., 2024] independently uses the same Elias-Fano structure as their succinct storage layer, motivating the choice by implementation simplicity; our contribution is the empirical quantification of the speed gap (Table 2.5).

2.1.5 Block sizes and Lookup strategy

Block occupancy under uniform distribution. The Exact Range Emptiness structure divides the universe into $B = 2^{\lfloor \log_2 n \rfloor}$ blocks; each non-empty block stores its keys' suffixes in a packed array (a *bucket*). The expected number of keys per block is $\lambda = n/B \in [1, 2)$. When n is a power of two, $B = n$ and $\lambda = 1$ exactly. Each key falls into a uniformly random block, so the occupancy of a fixed block follows $\mathrm{Bin}(n, 1/B)$, which converges to $\mathrm{Poisson}(\lambda)$ as $n \to \infty$ [Mitzenmacher and Upfal, 2017, Ch. 5].

For $\lambda = 1$, 36.8% of blocks are empty. Among non-empty blocks, the size distribution is sharply concentrated on the smallest values:

- $\Pr[X = 1 \mid X \geq 1] = e^{-1}/(1 - e^{-1}) \approx 58.2\%$ (median of 1 key);
- $\Pr[X \leq 2 \mid X \geq 1] \approx 87.3\%$;
- $\Pr[X \leq 3 \mid X \geq 1] \approx 97.0\%$ ($P_{90} = 3$).

That is, the local lookup is at most a 3-element scan in 97% of non-empty buckets, with 58% degenerating to a single comparison.

The classical balls-into-bins analysis [Mitzenmacher and Upfal, 2017, Lemma 5.1] bounds the maximum load by $3 \ln n / \ln \ln n$ with probability $\geq 1 - 1/n$. At $n = 2^{30}$ this gives ≈ 21 keys per bucket. Table 2.3 reports the empirical maxima alongside the Poisson reference k^* — the smallest k with $B \cdot \Pr[\mathrm{Poisson}(1) \geq k] < 1$; all measured maxima are well below the Lemma 5.1 bound. Each observed max is the largest bucket over a single realisation of n uniform random 64-bit keys, taken across all B blocks.

n	empty (%)	observed max	k^*	$B \cdot \Pr[X \geq k^*]$	w	k^*-bucket footprint (B)
2^{20}	36.75	9	10	0.12	44	55
2^{24}	36.79	9	11	0.17	40	55
2^{27}	36.79	10	12	0.11	37	56
2^{30}	36.79	13	12	0.89	34	51

Table 2.3: Bucket statistics for n keys in $B = n$ bins ($\lambda = 1$), assuming a full 64-bit universe ($K = 64$). k^*: smallest k with $B \cdot \Pr[\mathrm{Poisson}(1) \geq k] < 1$. $w = K - \lfloor \log_2 n \rfloor$ is the suffix width. The last column gives the bit-packed size of a bucket of k^* keys ($k^* w/8$ bytes, rounded up). In ARE deployments, K and hence w are typically smaller, so the resulting buckets are even more compact.

The Poisson probability mass function $\Pr[X = k] = e^{-\lambda} \lambda^k / k!$ [Feller, 1968, eq. (6.1)] makes the tail decay like $1/k!$: even at $B = 2^{30}$, the expected number of blocks with ≥ 20 keys is

$\sim 1.7 \times 10^{-10}$, making buckets larger than $\sim$20 keys virtually impossible for uniform data.

Non-uniform distributions and worst-case bounds For uniform data the previous paragraph showed that bucket sizes concentrate around $\lambda = 1$: empirical maxima stay below $\sim$13 keys even at $n = 2^{30}$ (Table 2.3), regardless of n.

Real-world data is rarely uniform. The original ARE construction [Goswami et al., 2015] uses a *locality-preserving* hash: consecutive input keys map to consecutive hashed values, so dense input regions stay dense in the hashed universe. On heavily clustered benchmarks the heaviest bucket reaches $\sim 6 \times 10^4$ keys. At the typical Facebook UDB range ($\mathcal{L} = 16$), median bucket sizes stay in the low thousands; near production P90 ($\mathcal{L} = 256$) they grow into the tens of thousands (Table 6.4, Chapter 6). The worst-case footprint stays at $\sim$120 KB — comfortably L2-cache-resident.

Regardless of distribution, the bucket size is hard-bounded above by the block capacity 2^w, where $w = K - \lfloor \log_2 n \rfloor$ is the suffix length: each bucket physically holds at most 2^w distinct w-bit suffixes. The choice of w is constrained by the overall filter size: the total ARE filter occupies approximately $n \cdot (w + 2)$ bits (w bits of suffix data plus $\sim$2 BPK of metadata). Compared to honest storage of 64-bit keys ($64n$ bits), the filter is meaningfully smaller only for w well below 64. In published range filter evaluations the total per-key budget falls in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1, Chapter 6). After subtracting the $\sim$2 BPK of rank/select metadata, this corresponds to a suffix width $w \in [12, 15]$ bits in our pipeline. A side benefit of $w = 16$ is that 16-bit suffixes align with SIMD lanes for vector-friendly comparisons.

At $w = 15$ the hard upper bound is $2^{15} = 32\,768$ keys per bucket. On a fully packed bucket of 2^w keys binary search terminates in at most $\log_2(2^w) = w$ iterations — and at $w = 15$ this still costs only $\sim$48 ns on Apple M4 Max, comparable to a single Rank/Select on the metadata bit vector. The bound is distribution-independent and depends only on w, not on n. In real workloads observed buckets are typically two orders of magnitude smaller than this hard limit; the worst case is bounded but rare.

By the way, distributions with dense ranges can be handled by the hybrid filter of Chapter 5: each cluster is isolated into its own exact sub-filter, turning local density into a compression advantage.

Search strategy. We implement both binary search $O(\log k)$ and linear scan $O(k)$ over the k suffixes of a bucket, and pick at query time based on k. Apple M4 Max benchmarks on uniform random w-bit suffixes: linear scan is fastest for small buckets ($\sim$3 ns at $k = 12$, $\sim$5 ns at $k = 24$, $\sim$10 ns at $k = 64$), while binary search stays cheap even for pathologically large buckets ($\sim$40 ns at $k = 2^{12}$, $\sim$60 ns at $k = 2^{17}$) — logarithmic growth compresses the worst case to tens of nanoseconds.

The default adaptive dispatch (linear for $k \leq 128$, binary above) gives the best of both branches: near-optimal in the common case and graceful degradation on adversarial distributions.

For uniform 64-bit keys with non-empty queries, the total ERE query takes $\sim$33–36 ns on Apple M4 Max, stable across $n \in [2^{20}, 2^{28}]$. Bucket search contributes only $\sim$3 ns at the typical $k \approx 12$; the remainder is Rank/Select navigation on the metadata bit vectors.

2.1.6 Weak Prefix Search

Definition. A *weak prefix query* on a sorted set $S' \subseteq U'$ is specified by a bit string p of length at most $\log_2 |U'|$ and returns the interval of ranks of points in S' whose binary representation has p as a prefix [Goswami et al., 2015, §3.2]. If no such points exist, the answer is arbitrary — hence *weak*. The supporting index answers the query in $O(1)$ time in the word-RAM model.

Use in ERE. Given a query range $[a, b]$ inside a single block, the original construction [Goswami et al., 2015, §3.2] reduces local emptiness to two weak prefix queries. Let p be the longest common prefix of the binary representations of a and b (an $O(1)$ most-significant-bit operation). Then $S' \cap [a, b] \neq \emptyset$ iff at least one of the following holds:

- the largest point in S' prefixed by $p \cdot 0$ exists and is $\geq a$, or
- the smallest point in S' prefixed by $p \cdot 1$ exists and is $\leq b$.

Each side is one WPS query plus a comparison, giving $O(1)$ local queries. The index is realised as a Z-Fast trie keyed by a Monotone Minimal Perfect Hash Function [Belazzougui et al., 2009].

Why we don't use it. The $O(1)$ guarantee is in the word-RAM model where hash table lookups and trie navigation cost $O(1)$ and space is expressed up to $O(n)$ additive terms. The hidden constants dominate in practice.

We implemented three WPS variants, pairing a Z-Fast trie (standard or succinct) with a Monotone Minimal Perfect Hash Function (MMPHF; fast or compressed):

	Space	Query latency (ns) at $n =$				
Variant	(bits/key)	2^{10}	2^{13}	2^{15}	2^{18}	2^{20}
Baseline (standard trie + fast MMPHF)	179	307	515	557	580	575
Compact (succinct trie + fast MMPHF)	151	490	681	712	751	812
Learned (succinct trie + learned MMPHF)	59	544	683	758	787	889

Table 2.4: Three WPS variants on Apple M4 Max, $K = 64$, 64-bit keys. Query latency generally rises with n because the trie deepens, so each query traverses more levels and pays one hash-probe-and-pointer-chase per level. Raw bench output: `research/wps/locators/wps_query_latency_2026-05-03.txt`.
TODO: re-measure at the thesis anchor scales $n \in \{2^{20}, 2^{24}, 2^{28}\}$ (current sweep stops at 2^{20}); other n va

Even the most compact variant costs 59 BPK — and this is an auxiliary index *on top of* the packed suffix array, which WPS accelerates but does not replace. In our use case (ERE inside an ARE filter, Section 4.1), the suffix width w is typically 7–13 bits per key depending on the target query range length $\mathcal{L}$, so WPS adds 5–8× more space than the suffix data it indexes.

Query latency is equally problematic. WPS replaces only the local search inside a bucket; Rank/Select navigation on the metadata bit vectors (∼27 ns) is identical for both approaches. Table 2.5 compares the bucket-search step alone across representative bucket sizes.

Bucket size k	Binary search (ns)	WPS Compact (ns)	Speedup
$\sim$12 (typical, uniform)	5–9	490–812	54–160$\times$
2^{12} ($k \approx 4{,}000$)	$\sim$40	490–812	12–20$\times$
2^{17} ($k \approx 131{,}000$)	$\sim$60	490–812	8–14$\times$

Table 2.5: Bucket-search latency on Apple M4 Max, $K = 64$, 64-bit keys. WPS Compact numbers are the full range over $n \in \{2^{10}, \ldots, 2^{20}\}$ from Table 2.4; binary search numbers are from the bucket-scan benchmark of Section 2.1.3. WPS pays a large per-call constant from trie pointer-chasing and hash probing that does not amortise with k, while binary search on packed suffixes scales as $O(\log k)$ with small constants on contiguous memory. TODO: double-check together with Table 2.4 before submission — the bucket-size axis here is conceptua

Even on adversarial bucket sizes WPS stays at least 8$\times$ slower for the operation it replaces.

The root cause is that the $O(1)$ theoretical guarantee hides large constants: each "constant-time" trie step hashes a variable-length prefix, probes a hash table, and chases a pointer. The probe and the pointer chase are random memory accesses. A WPS query traverses several trie levels and pays this cost on each, whereas a linear scan over a $\leq$13-element bucket ($\leq$55 B at $K = 64$, Table 2.3) touches a single cache line. Binary search on packed suffixes also requires no auxiliary index, whereas the most compact WPS variant adds 59 BPK on top of the suffix data.

2.1.7 Space Analysis

For K-bit keys, ERE stores $w = K - \lfloor \log_2 n \rfloor$ bits of suffix per key plus $\sim$2.53 BPK of metadata on the one-vector layout (single bit vector D with rank/select indices, Section 2.1.4; physical figure from Table 2.1, stable across n). At $K = 64$ and $n = 2^{28}$ ($w = 36$) this gives $\sim$39 BPK total — a $\sim$39% reduction over honestly storing the full 64-bit keys (64 BPK), but still $\sim$2.5$\times$ above the 14–16 BPK budget of industrial range filters (Table 6.1, Chapter 6).

Space grows linearly with K — unavoidable for an exact structure, since it must store the information distinguishing the keys. This linear dependence motivates the move to approximate range emptiness: by storing hashed fingerprints of length $K \approx \log_2(\mathcal{L}/\varepsilon)$ instead of full keys, the space becomes independent of the original key length.

2.1.8 Complexity Summary

Here W denotes the machine word size.

- **Build:** $O(n \cdot K)$ — a single pass over sorted keys.
- **Query:**

  Expected (uniform data): $O(\frac{K}{W})$ — block index lookup dominates; bucket size is $O(1)$ on average (Section 2.1.5).

  Worst case (all keys collide in a single block): $O(\frac{K}{W} \log n)$.

  Inside ARE with K-bit fingerprints fitting in one machine word ($K \leq W$): reduces to $O(1)$ expected.
- **Space:** $n(K - \log_2 n) + O(n)$ bits, matching the information-theoretic lower bound.

Chapter 3

Key Distributions

3.1 Key Distributions in Practice

The theoretical bounds of Section 1.4.1 hold for any key set. In practice, keys in storage systems follow distributions shaped by the application domain.

3.1.1 Common Industrial Distributions

Keys in LSM-tree storage systems typically exhibit one or more of the following patterns:

Sequential / near-sequential. Auto-incrementing primary keys, monotonic timestamps, log sequence numbers. Keys are approximately evenly spaced with small gaps. The key space is dense – the spread is much smaller than the universe. Among the Search on Sorted Data (SOSD) [Kipf et al., 2019] datasets, Facebook (user IDs), Books (popularity keys), and Wiki (edit timestamps) all fall into this category.

Clustered. Keys concentrate in a few dense regions separated by large gaps. Event logs keyed by user ID prefix + timestamp cluster around active users; inactive users leave gaps. Geospatial keys such as S2 cell IDs [Google, 2017] cluster around populated areas, with sparse rural regions. OpenStreetMap (OSM) cell IDs are a representative example: they span nearly the full 64-bit space, yet most keys concentrate in a small fraction of it.

Uniform-like. UUIDs, cryptographic hashes, randomly generated tokens. Keys are spread across the full key space with no exploitable structure. This is the hardest case for distribution-aware optimizations.

3.1.2 SOSD Benchmark Datasets

For empirical evaluation, we use real-world datasets from the SOSD (Search on Sorted Data) benchmark [Kipf et al., 2019]:

Dataset	n (full)	Character
Facebook	200M	User IDs from Facebook's social graph (uint64).
Books	200M	Amazon book popularity keys (uint32).
Wiki	200M	Unix timestamps of Wikipedia article edits (uint64).
OSM	800M	Hierarchical cell IDs covering global OpenStreetMap data (uint64).

Table 3.1: SOSD datasets used in our evaluation. Benchmarks in Chapter 6 use subsets of these datasets (e.g. $n = 2^{24}$)

Remark 4. *All datasets are sorted and deduplicated on load. The Wiki dataset contains many repeated timestamps, so its effective n is smaller than the requested raw count.*

Dataset	n	U [bits]	P10	P50	P90	max	mean
Facebook	≈200M	37	2	28	693	5.7×10^5	387
Wiki	≈90M	29	1	1	3	1.7×10^5	3
Books	≈200M	31	1	2	9	2.0×10^4	5
OSM	800M	64	1.6×10^6	4.5×10^7	1.7×10^9	2.3×10^{17}	1.7×10^{10}

Table 3.2: Consecutive-key gap statistics for SOSD datasets after sort and deduplication. n: effective key count. U: key range in bits ($\lfloor \log_2(\max - \min) \rfloor + 1$). Gaps are differences between adjacent sorted keys.

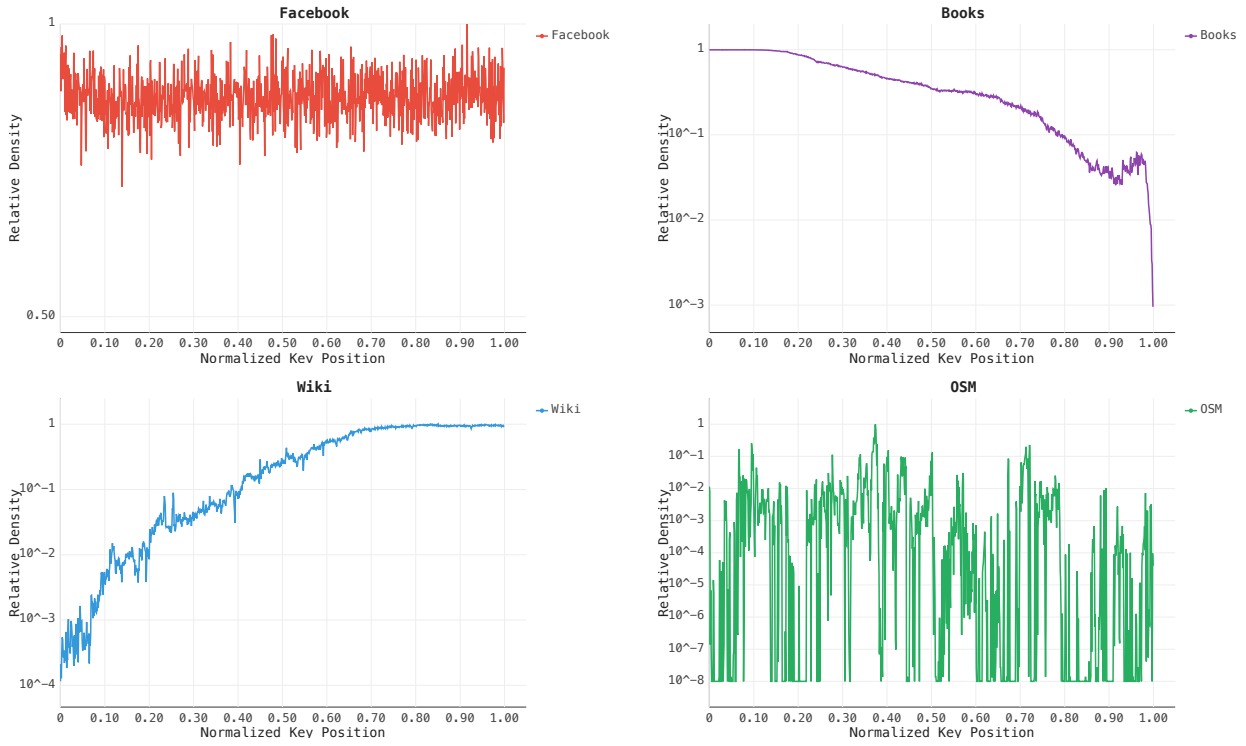

Figure 3.1: Key density histograms for SOSD datasets (1000 bins, log-scale Y-axis). Relative density is the fraction of all keys falling in each bin, normalized so that a perfectly uniform distribution gives 1 everywhere.

3.1.3 Synthetic Distributions

To isolate the effect of specific distribution properties, we also generate synthetic key sets:

- **Uniform:** keys drawn uniformly from $[0, 2^{60})$. Average gap $\approx 2^{60}/n$. No clustering.
- **Clustered:** a mixture of Gaussian clusters with controlled parameters (number of clusters, intra-cluster spread, inter-cluster gap).

Together, the SOSD and synthetic datasets span the spectrum from near-sequential (Facebook, Books, Wiki) through clustered (OSM) to structureless (uniform random) — allowing us to evaluate each filter design across the range of distributions it may encounter in practice.

3.1.4 An assumption on queries

We assume that queries target regions of the key space where stored keys are dense, rather than arbitrary empty regions.

Assumption 1. *Queries reflect the application state and therefore concentrate on regions that contain data.*

Consequently, FPR in sparse regions of the key space matters less — dense regions dominate the effective FPR experienced by the application.

The SODA hash (Section 4.1) ignores this assumption entirely — it provides worst-case guarantees for any distribution. The truncation hash (Section 4.2) performs well only under uniformity. The hybrid approaches (Chapter 5) explicitly exploit it by isolating dense regions.

Section 3.2 operationalizes this assumption in the query workload.

3.2 Query generation

All benchmarks measure FPR on *empty* queries — range queries $[a, b]$ that contain no stored key, so every positive answer is a false positive by definition.

Queries are drawn from a three-bucket mixture that reflects the assumption of Section 3.1.4:

- **50% near-key.** A random stored key k is chosen; the query start is drawn uniformly from $[k - 5L,\ k + 5L)$ and clamped to the nearest gap. Models application lookups that miss by a small margin.
- **30% in-gap.** A random gap between two adjacent keys is selected uniformly; the query is placed entirely inside it. Models misses in locally dense regions.
- **20% uniform.** A random start is drawn uniformly from $[\min k, \max k]$ and clamped to the nearest gap. Covers sparse regions to verify FPR stays reasonable there too.

The near-key and in-gap buckets reflect the dominant workload; the uniform bucket acts as a safety check on sparse regions.

When a candidate query $[a, a+L)$ would contain a stored key k^*, it is *truncated* to $[a, k^*-1]$ rather than discarded and resampled. Truncation preserves queries that start inside a cluster

and end between two closely spaced keys, which would otherwise be rejected every time — biasing the workload towards cluster boundaries and missing the harder case of a query landing deep inside a dense region. The effective range length may therefore be smaller than L.

Chapter 4

Hash Function Approaches

4.1 Pairwise-Independent Hash (SODA Baseline)

4.1.1 The Hash Function

The paper's original locality-preserving hash [Goswami et al., 2015, §3.1]:

$$h(x) = \left(u(\lfloor x/2^K \rfloor) + (x \bmod 2^K)\right) \bmod 2^K \tag{4.1}$$

That is: compute the block index $\lfloor x/2^K \rfloor$, hash it with u to get a per-block rotation, then add the key's within-block offset $x \bmod 2^K$. The result is in $[0, 2^K)$.

$u : [0, U/2^K) \to [0, 2^K)$ is drawn from the multiply-add-shift family of Dietzfelbinger [1996], stored as two 64-bit coefficients (a, b):

$$u(x) = \left\lfloor \frac{a \cdot x + b \bmod 2^{64}}{2^{64-K}} \right\rfloor. \tag{4.2}$$

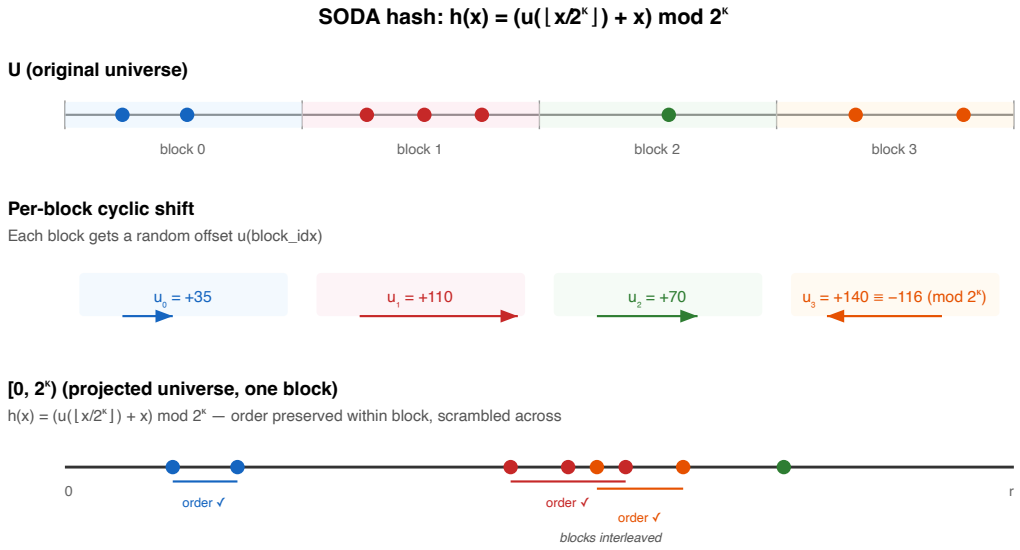

Figure 4.1: SODA hash mechanism: per-block rotation maps keys to $[0, 2^K)$. A query range spanning two blocks produces at most two intervals in the mapped space.

Properties of the hash:

- **Pairwise-independent:** $\Pr_{h\sim\mathcal{H}}[h(x_1) = h(x_2)] \leq 2^{-K}$ for every pair $x_1 \neq x_2 \in U$, where $\mathcal{H}$ is the multiply-add-shift family (Equation (4.2)). This holds for sequential, clustered, or adversarial data.
- **Locality-preserving:** the hash applies a per-block rotation, so consecutive keys within the same block map to consecutive positions in $[0, 2^K)$. When $\mathcal{L} \leq 2^K$, a query range $[a, b]$ spans at most 2 blocks, producing at most 2 intervals in $[0, 2^K)$, each checked with one IsEmpty call to ERE. (The choice $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil \geq \log_2 \mathcal{L}$ guarantees this.)

Comparison with Grafite. Grafite [Costa et al., 2024] uses a hash of the same locality-preserving form but from a different pairwise-independent family: $q(x) = \big((c_1 x + c_2) \bmod p\big) \bmod r$, where $p = 2^{61} - 1$ is a Mersenne prime. This construction requires modulo operations with non-power-of-two divisors (p and r), whereas the multiply-add-shift family used here [Dietzfelbinger, 1996] replaces them with a multiplication and a right shift, since all reductions are by powers of two. The theoretical guarantees are identical: pairwise independence gives FPR $\leq \varepsilon$ for any data and query distribution.

4.1.2 ARE Construction With SODA Hash

This is a direct implementation of the construction from Goswami et al. [2015, §3]. The hash design, its proofs of locality-preservation and pairwise independence, and the resulting space/FPR bounds are all from the original paper — our contribution is the practical implementation.

Combining this hash with ERE (Chapter 2) yields the baseline ARE filter:

1. Divide U into blocks of size 2^K. Let $\text{blockIdx}(x) = \lfloor x/2^K \rfloor$ be the block index of key x.
2. For each block, compute pairwise-independent shift: top K bits of $(a \cdot \text{blockIdx}(x) + b)$.
3. Hash each key via Equation (4.1).
4. Sort, deduplicate, build ERE over $[0, 2^K)$.
5. Query: hash both endpoints; rotation may split the range into two intervals — check both in ERE.

Filter properties.

- **FPR $\leq \varepsilon$ for any data distribution** — follows from pairwise independence of the hash, with $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ giving BPK $= \log_2(\mathcal{L}/\varepsilon) + O(1)$, matching the information-theoretic lower bound. Including ERE metadata, the total cost is $\log_2(\mathcal{L}/\varepsilon) + \sim 2$ bits per key.
- **No false negatives** — if key $x \in [a, b]$, then $h(x)$ lies in the mapped range $h([a, b])$, which is fully checked in ERE.

4.1.3 Empirical Validation

On uniform data ($n = 2^{20}$, $\mathcal{L} = 128$), the measured FPR-vs-BPK curve tracks the theoretical bound with a gap of $\sim$2 BPK (ERE overhead, Figure 4.2).

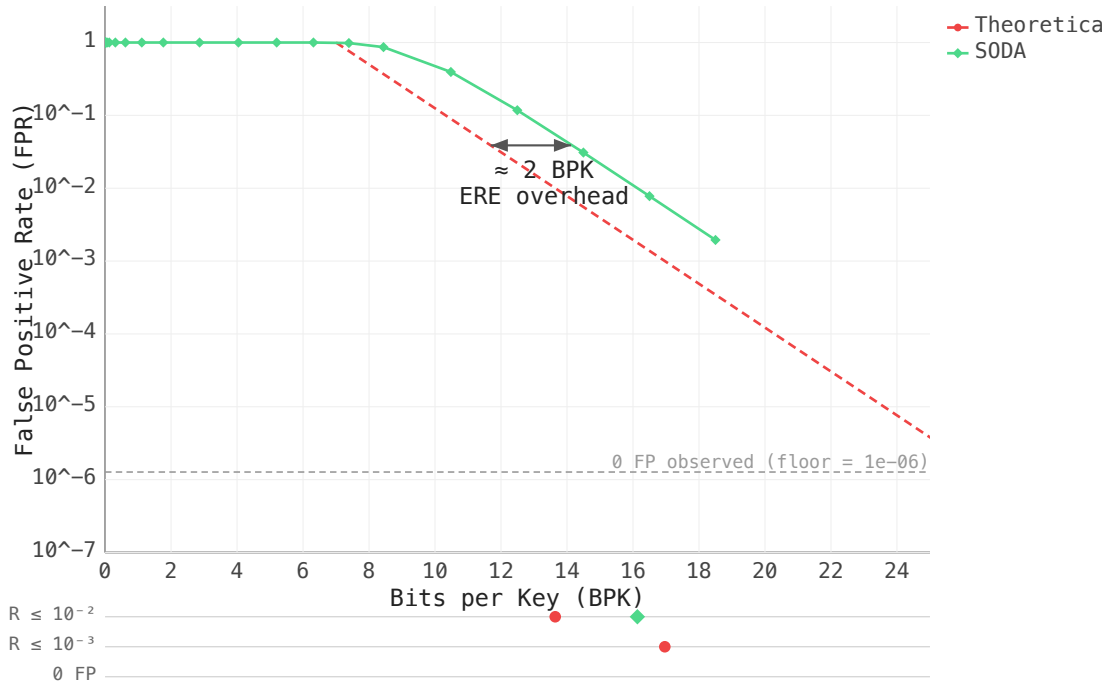

Figure 4.2: FPR vs BPK on uniform keys ($n = 2^{20}$, $\mathcal{L} = 128$). SODA tracks the theoretical bound with ∼2 BPK overhead from ERE metadata.

4.2 Prefix Truncation

4.2.1 The Hash Function

ERE size grows with the bit-length of keys. We need to compress keys into a smaller range $[0, 2^K)$ while preserving locality – so that a query interval maps to a bounded number of intervals in the compressed space. This need not be a hash in the cryptographic sense; the simplest locality-preserving compression is plain bit-truncation:

$$h(x) = \lfloor x/2^t \rfloor,$$

keeping the top $K = L - t$ bits of each key (where L is the key width in bits).

Properties of the hash:

- **Locality-preserving:** truncation preserves order — if $a < b$, then $h(a) \leq h(b)$. Intervals map to intervals.
- **Contiguous phantoms:** recall that the phantom points of a key x are all non-keys y with $h(y) = h(x)$ — the false pre-images that cause false positives. Under truncation they all fall within an interval of length 2^t in the original universe: exactly the non-keys sharing the same K-bit prefix as x. This contrasts with the SODA hash, where phantoms are scattered uniformly.

Remark 5. *Throughout this section, $2^t = 2^{L-K}$ is the universe compression factor; the number of original-universe points that hash to the same value in $[0, 2^K)$. For truncation this matches the definition: t bits are dropped from the back of each key.*

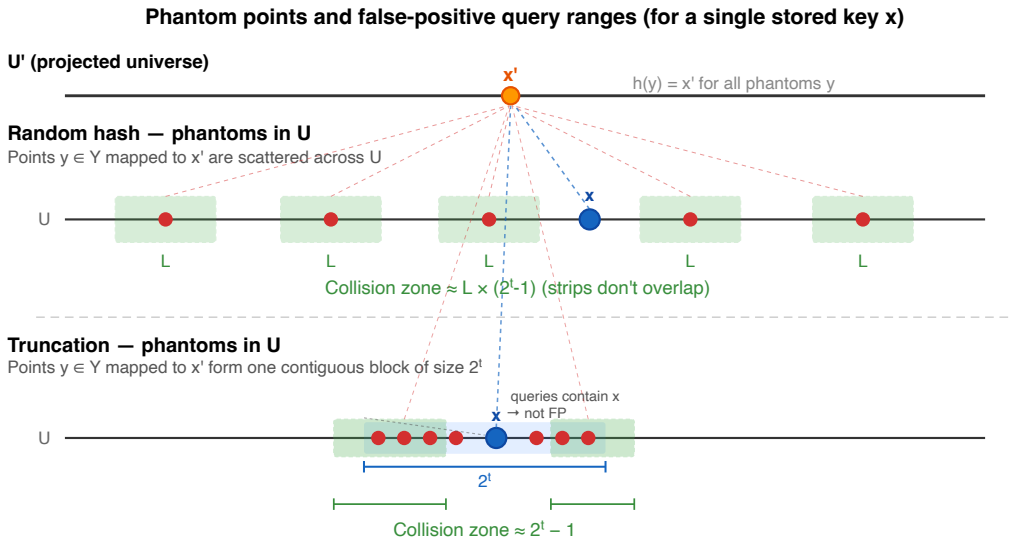

Figure 4.3: Phantom point distribution: random hash (SODA) scatters phantoms uniformly across U; truncation concentrates them in the contiguous 2^t-point block that contains the key.

4.2.2 ARE Construction With Truncation

Combining truncation with ERE yields a filter where the contiguous phantom structure (Figure 4.3) changes the collision analysis:

- **SODA hash:** phantoms scattered $\Rightarrow$ collision zone per key is $\mathcal{L} \times (2^t - 1)$ (Section 4.1).
- **Truncation:** phantoms contiguous in one block $\Rightarrow$ an empty query overlaps a key's phantom block in $2^t - 1$ positions on average, giving collision zone $2^t - 1$ (Eq. (4.3)).

The additive (rather than multiplicative) interaction eliminates the $\mathcal{L}$ factor from the filter's space formula:

Hash	Collision zone	K for FPR $\leq \varepsilon$	Filter BPK
Random (SODA)	$\mathcal{L} \times (2^t - 1)$	$\log_2(n\mathcal{L}/\varepsilon)$	$\log_2(\mathcal{L}/\varepsilon) + O(1)$
Truncation	$2^t - 1$	$\log_2(n/\varepsilon)$	$\log_2(1/\varepsilon) + O(1)$

Table 4.1: Choosing truncation over the SODA hash reduces the resulting filter's BPK by $\log_2(\mathcal{L})$. At $\mathcal{L} = 128$, this is a saving of 7 bits per key.

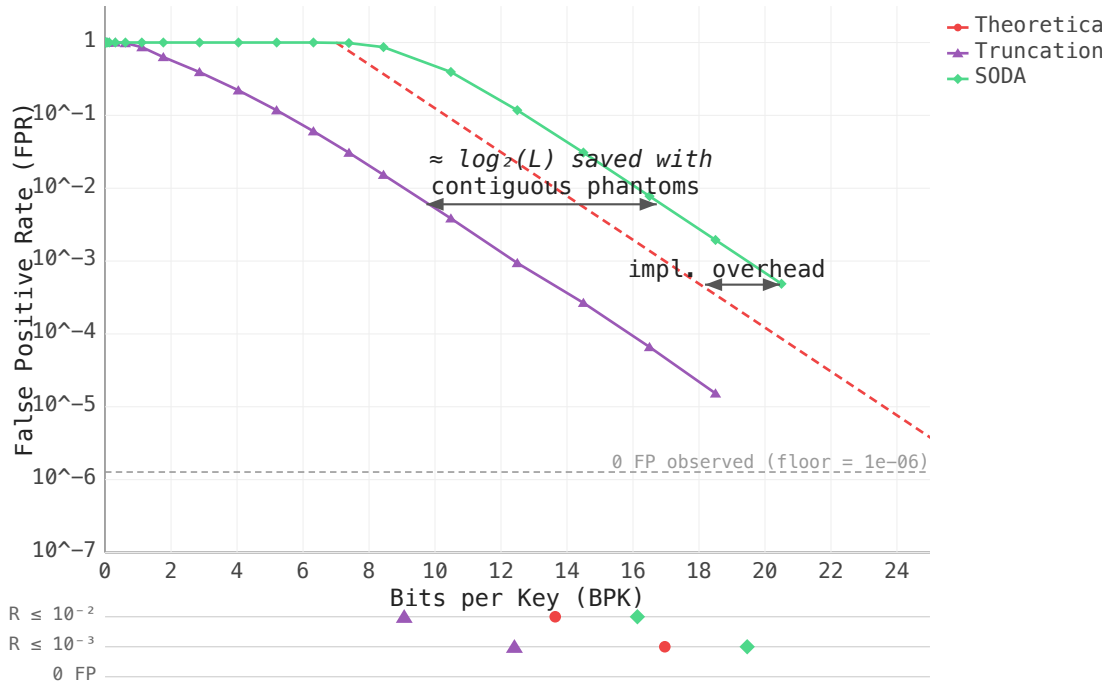

Figure 4.4: FPR vs BPK for truncation vs SODA on uniform keys ($n = 2^{20}$, $\mathcal{L} = 128$). The SODA curve is shifted right by $\approx \log_2(128) = 7$ BPK — matching the theoretical prediction.

4.2.3 The Price of Contiguity

The same contiguity that saves space is also the weakness.

FPR for Uniform Key Spacing

Let R denote the uniform gap between consecutive keys. Assume keys x_0, $x_0{+}R$, $x_0{+}2R, \ldots$ A query $[q,\ q{+}\mathcal{L}{-}1]$ is truly empty iff it fits inside a gap: $q = x_i + s$, $s \in [1,\ R{-}\mathcal{L}]$.

A false positive requires some key's hash value to fall inside $h([q, q+\mathcal{L}-1])$. Since the query is empty, every block *fully covered* by the query is key-free: any key inside such a block would lie in the query and contradict emptiness. Only the two *edge* blocks — the one straddled by q on the left and the one straddled by $q + \mathcal{L} - 1$ on the right — can hold a key whose hash collides with $h([q, q + \mathcal{L} - 1])$, and it suffices to consider the immediate neighbours x_i and x_{i+1}: order-preservation of h gives $h(x_{i+2}) \geq h(x_{i+1})$, so if x_{i+2} collides with the query then x_{i+1} does too — further keys cannot trigger *new* false-positive events (symmetrically on the left). The argument holds regardless of how many blocks the query spans.

Let $\alpha_i = x_i \bmod 2^t$ and $\alpha_{i+1} = x_{i+1} \bmod 2^t$ be the offsets of the neighbours within their respective blocks.

- **Left overlap:** the query overlaps x_i's phantom block iff $q \leq x_i + 2^t - 1 - \alpha_i$, i.e., $s \leq 2^t - 1 - \alpha_i$ — contributing $2^t - 1 - \alpha_i$ values of s.
- **Right overlap:** the query overlaps x_{i+1}'s phantom block iff $q + \mathcal{L} - 1 \geq x_{i+1} - \alpha_{i+1}$, i.e., $s \geq R - \mathcal{L} + 1 - \alpha_{i+1}$ — contributing α_{i+1} values of s.

When $R - \mathcal{L} \geq 2^{t+1}$, the two ranges of s are disjoint, so the false-positive count per gap

is $(2^t - 1 - \alpha_i) + \alpha_{i+1}$. Treating α_i, α_{i+1} as uniform on $\{0, \ldots, 2^t - 1\}$ (mean $(2^t - 1)/2$) and dividing by the $R - \mathcal{L}$ empty-query positions:

$$\text{FPR} \;=\; \frac{2^t - 1}{R - \mathcal{L}} \;\approx\; \frac{2^t}{R - \mathcal{L}}. \tag{4.3}$$

Capping at 1 and handling the $R \leq \mathcal{L}$ case:

$$\text{FPR} = \begin{cases} 0 & R \leq \mathcal{L} \text{ (no empty queries)}, \\ \min\bigl(1,\ 2^t/(R - \mathcal{L})\bigr) & R > \mathcal{L}. \end{cases}$$

Dense or sequential keys ($R \leq \mathcal{L}$) have FPR = 0 for free: every query of length $\mathcal{L}$ hits a key, so the filter is exact. Low FPR requires $R - \mathcal{L} \gg 2^t$; when $R - \mathcal{L} \lesssim 2^t$ the phantom block covers most of the gap and FPR approaches 1.

Breakdown thresholds for SOSD datasets

From Eq. (4.3), high FPR occurs when $2^t \gtrsim R - \mathcal{L}$, so the critical truncation depth is $t^* = \lfloor \log_2(R - \mathcal{L}) \rfloor$. Table 4.2 shows t^* for typical SOSD gaps and several query lengths.

Dataset	Typical R	$\mathcal{L} = 4$	$\mathcal{L} = 16$	$\mathcal{L} = 128$	$\mathcal{L} = 1024$
Wiki / Books	1–5	0*	0*	0*	0*
Facebook P50	28	4	3	0*	0*
Facebook mean	387	8	8	8	0*
OSM P50	4.5×10^7	25	25	25	25

* $R \leq \mathcal{L}$: every query contains a key.

Table 4.2: Critical truncation depth t^* at which FPR $\to 1$ for SOSD gap statistics (Table 3.2) and varying query length $\mathcal{L}$.

Wiki and Books have gaps well below any practical $\mathcal{L}$, so every query hits a key. Facebook's tail (gaps up to ∼700) breaks at $t \approx 8$. OSM has $t^* \approx 25$ for all query lengths. In a 64-bit universe this means keeping $K = 64 - 25 = 39$ bits just to avoid complete failure; ERE compresses these to $K - \log n \approx 19$ BPK at $n = 2^{20}$. Achieving FPR $\leq 10^{-3}$ requires $t \leq 15$ ($K \geq 49$, ≈ 29 BPK) under uniform queries. Thus, truncation is impractical for clustered data.

Remark 6. *The derivation assumes* ***queries are placed uniformly at random in the universe****, so α_i, α_{i+1} are uniform on $\{0, \ldots, 2^t - 1\}$. Benchmark queries (Section 3.2) are drawn from the gaps between keys, so they land near phantom zones more often than a uniform draw would.*

The SODA hash avoids this entirely: pairwise independence gives FPR $\leq \varepsilon$ for any key distribution, at the cost of $\log_2(\mathcal{L})$ extra bits per key.

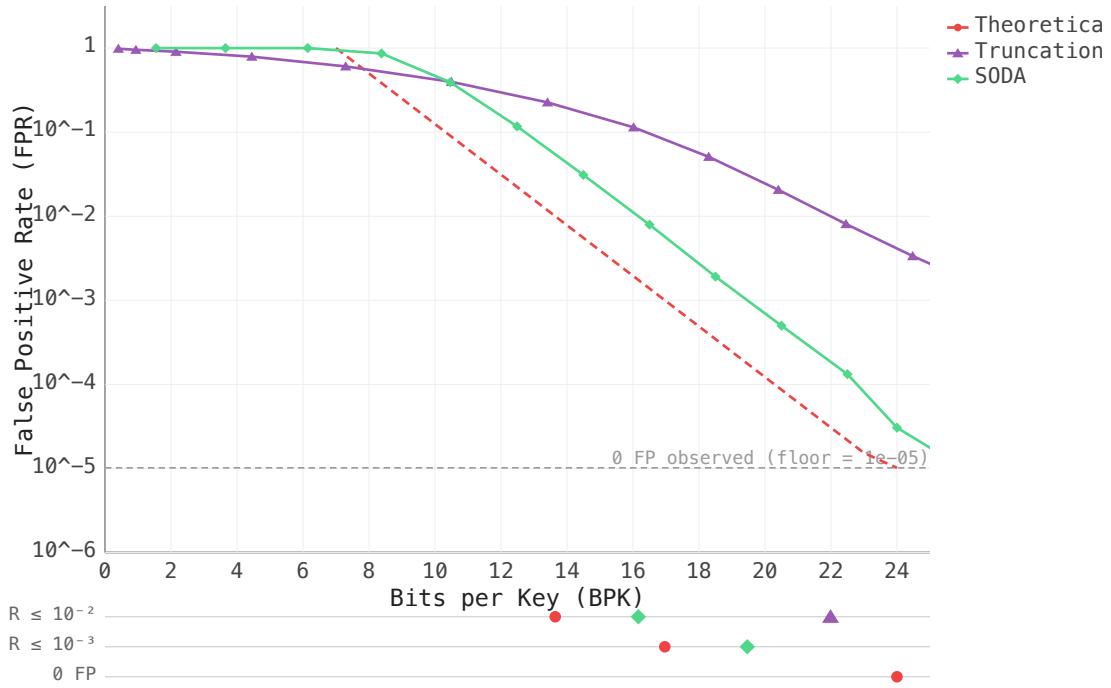

Figure 4.5: FPR vs BPK on OSM ($n = 2^{20}$, $\mathcal{L} = 128$, in-gap queries, Section 3.2). Truncation is bounded below by a structural FPR floor set by the gap-to-2^t ratio (Eq. 4.3): at 24 BPK it remains at FPR $\approx 10^{-2}$ while SODA reaches $\sim 10^{-5}$.

Remark 7. *Grafite [Costa et al., 2024] independently proposes an equivalent heuristic named* Bucketing *(their Section 4): keys are mapped via $x \mapsto \lfloor x/s \rfloor$ and the occupied bucket indices are stored in Elias-Fano, with s as the space-accuracy trade-off parameter — the same idea as prefix truncation with $s = 2^t$. Our contribution relative to Grafite is the precise FPR formula (Equation 4.3) and the per-dataset breakdown analysis (Table 4.2), which Grafite does not provide.*

4.3 Adaptive Filter

The SODA hash compresses keys into a K-bit universe of size $2^K = n\mathcal{L}/\varepsilon$. If the original key spread $\max(S) - \min(S)$ is already smaller than 2^K, no hash is needed at all — we build ERE directly on the (normalized) keys and get FPR = 0 for free. The adaptive filter checks this condition at build time.

4.3.1 Exact Mode

1. **Normalize:** subtract $\min(S)$ from all keys, shifting the dataset to start at 0.
2. **Measure spread:** $M = \lceil \log_2(\max(S) - \min(S)) \rceil$.
3. **Decide:**
   - $M \leq K$: the entire dataset fits in K bits ⇒ **exact mode**. Build ERE directly over normalized keys. No hash, FPR = 0.
   - $M > K$: data too spread ⇒ **SODA mode**. Apply a hash drawn from a pairwise-independent family, same filter guarantees as Section 4.1.

4.3.2 When Does Exact Mode Trigger?

Rewriting the condition

$$\max(S) - \min(S) < 2^K = \frac{n\mathcal{L}}{\varepsilon}$$

in terms of the average gap:

$$\bar{g} = \frac{\max(S) - \min(S)}{n-1}$$

$$\bar{g} \; < \; \frac{\mathcal{L}}{\varepsilon}. \tag{4.4}$$

At $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$ the threshold is 128,000 — well above the Facebook median gap of 28 or the Books mean of 5, so exact mode triggers on these datasets for free.

This becomes especially powerful in combination with hybrid segmentation (Chapter 5): each dense cluster is isolated into its own adaptive filter, where the local spread is small enough for exact mode, giving FPR = 0 on the dense parts of the data.

Chapter 5

Hybrid Segmentation

Chapter 3 identified three distribution types. Sequential and near-sequential keys have small local spread, so exact mode (Section 4.3) applies directly — FPR = 0 with no hash needed. When keys are uniform over a large span, truncation (Section 4.2) is effective: large inter-key gaps mean few phantom collisions and low FPR.

Clustered keys (OSM, geospatial IDs) fit neither case alone: dense regions inside each cluster behave like near-sequential data, while the sparse gaps between clusters have large inter-key distances where truncation works well.

The hybrid idea is to split along this structure — isolate each dense region into its own exact-mode sub-filter (FPR = 0), and route the sparse keys between clusters to a fallback filter.

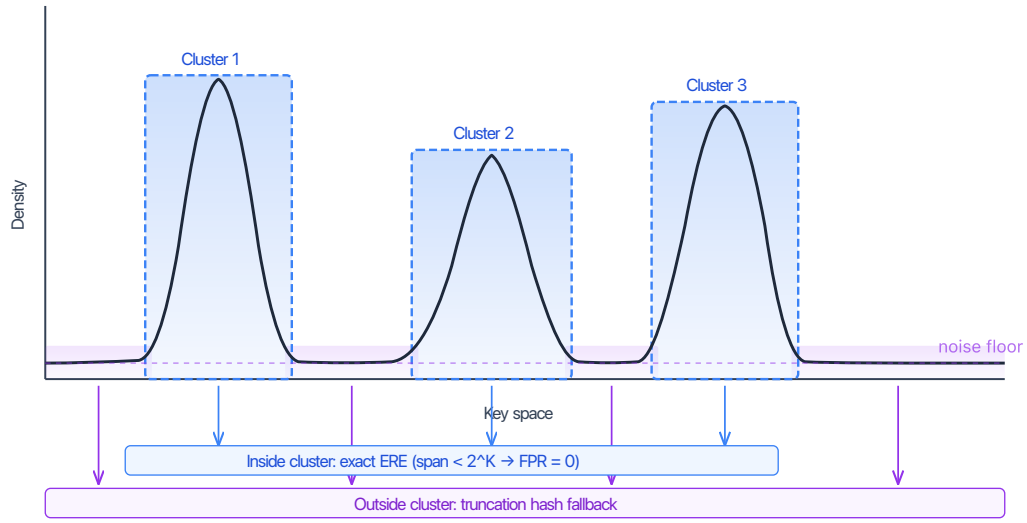

Figure 5.1: Hybrid segmentation: dense clusters go into exact-mode sub-filters (FPR = 0); sparse keys between clusters go to a fallback.

This chapter describes SegARE, our segmentation algorithm based on 1D DBSCAN.

5.1 1D DBSCAN in $O(n)$

DBSCAN [Ester et al., 1996] is a density-based clustering algorithm. It takes two parameters: a neighbourhood width δ and a density threshold `minPts`. A point is a *core point* if it has at least `minPts` neighbours within a window of width δ; a *border point* if it is within δ of a core point but not itself dense; and *noise* otherwise. Clusters grow from core points by absorbing all density-reachable neighbours.

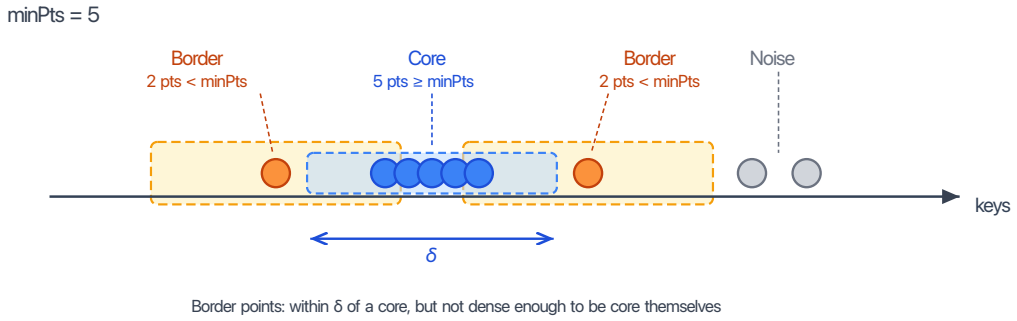

Figure 5.2: 1D DBSCAN on a key sequence (minPts = 5). Blue = core points (window contains $\geq$ minPts neighbours). Orange = border (within δ of a core, but $<$ minPts in own window). Gray = noise.

In a general metric space, finding the δ-neighbourhood of each point costs $O(n^2)$. On sorted one-dimensional data, density queries reduce to a sliding window, giving an $O(n)$ implementation.

5.2 SegARE: Design and Algorithm

Border expansion increases bits per key. Standard DBSCAN absorbs border points — points within δ of a core but not themselves dense — into the nearest cluster. This widens the cluster's key-space universe without adding proportional density. The exact-mode ERE sub-filter encodes keys relative to that universe, so a wider span means more bits per key. SegARE drops border expansion: the dense core alone minimises span and keeps the sub-filter cost low.

Expressing δ through `minPts`. Standard DBSCAN treats δ and `minPts` as independent. We choose δ small enough that every range produced by the forward sweep fits the exact-mode budget of the underlying ERE sub-filter (see Section 5.2.4 for the derivation):

$$\delta = \texttt{minPts} \cdot \frac{\mathcal{L}}{2\varepsilon} = \frac{256\,\mathcal{L}}{\varepsilon}. \tag{5.1}$$

This ties δ to `minPts`, leaving a single free parameter.

Choosing `minPts` = 512. The choice balances three concerns:

- **Overhead per key.** Each segment costs roughly 700 bits of fixed metadata (`minKey`, `maxKey`, pointer, sub-filter header). At 512 keys this adds ~ 1.4 BPK; for smaller clusters the overhead quickly dominates the budget.

- **Query cost.** The segment count is at most $\lfloor n/\texttt{minPts} \rfloor$, so the binary search over segment boundaries (Section 5.2.3) stays fast regardless of dataset size.
- **Detection power.** If `minPts` is set too high, genuinely dense sub-ranges with fewer keys go entirely to the fallback, negating the benefit of segmentation.

512 sits at a practical sweet spot: small enough to catch real clusters, large enough to keep per-segment overhead acceptable.

To sanity-check the scale: for $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$ we get $\delta \approx 3.3 \times 10^7$. Comparing this against the SOSD gap statistics (Table 3.2), Facebook/Wiki/Books have median gaps well below δ so most keys fall into segments, while OSM's median gap of 4.5×10^7 sits right at the threshold — segmentation captures only the denser sub-clusters. A uniform 64-bit distribution has average gap $\sim 2^{44} \gg \delta$, so every key goes to the fallback and SegARE degrades to plain truncation.

5.2.1 Algorithm

SegARE runs in three steps:

1. **Forward sweep:** slide a two-pointer window. Whenever the window holds $\geq$ `minPts` keys, record the index range [*left*, *right*].
2. **Conditional merge:** adjacent ranges with key-space gap $\leq \delta$ are joined only if the combined span is below $m'\mathcal{L}/\varepsilon$, where m' is the total key count of the merged segment. Otherwise they remain separate.
3. **Fallback:** all keys outside merged ranges go to the fallback filter.

5.2.2 Fallback

Keys not assigned to any segment go to a single fallback ARE filter. Depending on the density of the fallback keys, this may be a truncation filter or a hash-based one.

5.2.3 Query

Segments are sorted by $[\texttt{minKey}, \texttt{maxKey}]$. For a query $[a, b]$:

1. Binary search over segments intersecting $[a, b]$ — $O(\log C)$, C = segment count.
2. Check the first intersecting segment only. Suppose the query $[a, b]$ intersects two segments $S_1 = [\texttt{minKey}_1, \texttt{maxKey}_1]$ and $S_2 = [\texttt{minKey}_2, \texttt{maxKey}_2]$ with $\texttt{maxKey}_1 < \texttt{minKey}_2$. Then $a \leq \texttt{maxKey}_1$ and $b \geq \texttt{minKey}_2$, so the query covers $\texttt{maxKey}_1$ — a real key. The exact-mode sub-filter of S_1 reports non-empty, and no further segments need to be checked.
3. Check the fallback filter.
4. Return "empty" only if all agree.

5.2.4 FPR Guarantee

Every segment sub-filter operates in exact mode (FPR = 0).

Individual range. Consider a range of m sorted keys $k_0 < k_1 < \cdots < k_{m-1}$ from the forward sweep. The window constraint says any `minPts` consecutive keys span at most δ: $k_{i+(\texttt{minPts}-1)} - k_i \leq \delta$ for every valid i. We split the $m-1$ gaps into $\lceil (m-1)/(\texttt{minPts}-1) \rceil$ non-overlapping blocks of $\texttt{minPts}-1$ consecutive gaps; each block spans $\leq \delta$ by the window constraint, so:

$$k_{m-1} - k_0 \;\leq\; \left\lceil \frac{m-1}{\texttt{minPts}-1} \right\rceil \cdot \delta.$$

With $\delta = \texttt{minPts} \cdot \mathcal{L}/(2\varepsilon)$, the ceiling factor $\lceil (m-1)/(\texttt{minPts}-1) \rceil$ equals 1 for $m \leq \texttt{minPts}$ (span $\leq \delta < m\mathcal{L}/\varepsilon$, trivially satisfied) and first reaches 2 at $m = \texttt{minPts}+1$, where the margin is tightest:

$$2\delta \;=\; \texttt{minPts} \cdot \frac{\mathcal{L}}{\varepsilon} \;<\; (\texttt{minPts}+1) \cdot \frac{\mathcal{L}}{\varepsilon} \;=\; m \cdot \frac{\mathcal{L}}{\varepsilon}.$$

Hence every range from the forward sweep fits the exact-mode budget.

Merged segments. The merge step is conditional: two adjacent ranges are joined only if the combined span is still below $m'\mathcal{L}/\varepsilon$ for the combined size m'. This preserves the exact-mode invariant by construction.

False positives can therefore come only from the fallback filter, giving overall FPR $\leq \varepsilon$.

Chapter 6

Experimental Evaluation

6.1 Benchmark Setup

All experiments use 60-bit keys from the SOSD benchmark suite [Kipf et al., 2019]: Facebook user IDs, Wikipedia edit timestamps, OpenStreetMap cell IDs, and Books ISBNs. Synthetic distributions (uniform, clustered) supplement the real-world datasets.
TODO: 60 bits - SNARF overflow when values are close to UINT64_MAX, measure separatly

Competing implementations:

- **SegARE**: our simplified 1D DBSCAN hybrid filter (Section 5.2).
- **Greedy+Merge**: our greedy spread-threshold hybrid filter.
- **Grafite** [Costa et al., 2024]: information-theoretically optimal range filter (SIGMOD 2024).
- **SNARF** [Vaidya et al., 2022]: learned CDF-based range filter.
- **SuRF** [Zhang et al., 2018]: succinct range filter with three variants (base, hash suffix, real suffix).
- **Bloom ARE**: trivial baseline — standard Bloom filter with $\mathcal{L}$ point lookups per range query.

All filters are evaluated on FPR-vs-BPK tradeoff curves, measured over 10^6 random empty-range queries per configuration — enough to register 10^3–10^4 false positives at our target FPR range of 10^{-2}–10^{-3} (Section 6.2, "FPR targets").

6.2 Workload Parameter Choices

Range filters in LSM-tree storage systems are built *per SSTable*: each on-disk file carries its own filter, and the relevant input size n is the number of keys in one SSTable, not the database total. Production deployments keep SSTables small — tens to hundreds of megabytes — for several reasons listed below.

1. **Memory and time, compounded.** The build of the filter during compaction operations needs random access to keys, so the working set — not just the filter — must be held in RAM during the operation. Filter construction itself runs at a few million keys per

second: Costa et al. [2024] report ∼7 M keys/s for Grafite at $n = 2 \times 10^8$ on a single thread. Both RAM and time scale linearly with n; compacting an $n = 2^{30}$ SSTable claims gigabytes of RAM and minutes of CPU on a single operation. This single-thread cost is unavoidable: in LevelDB, Cassandra, HBase, and Pebble each compaction job is single-threaded by design; RocksDB's optional subcompactions partition a job across threads but emit a separate SSTable per subcompaction, so the filter for one output file is still built by one thread [Dong et al., 2021].

2. **Cloud I/O.** Loading and saving SSTables on object storage backends (S3, GCS) is bandwidth-bound at the per-object level: a single connection to S3 sustains ∼55–95 MiB/s end-to-end [Durner et al., 2023], so a multi-gigabyte file streamed to a remote VM takes tens of seconds. Splitting an object across parallel uploads scales aggregate throughput further but competes with the live database for network bandwidth and CPU.

3. **Blast radius.** Corruption of one SSTable file or one object loses exactly the keys in that file. Smaller files bound the blast.

4. **Sharding.** Multiple smaller SSTables can be queried in parallel for the on-disk scans following a filter probe, migrated independently between nodes, and replicated at file granularity; one giant file cannot. TODO: do we really need this point?

Realistic per-SSTable n. The default RocksDB `target_file_size_base` is 64 MB, with production deployments commonly tuning to 128 MB; entry sizes in real workloads (key + value) are roughly 60–150 bytes on average [Cao et al., 2020], putting the typical per-SSTable count at $n \in [2^{19},\ 2^{21}]$.

Two production cases push n higher. Secondary-index and counter column families (CFs — RocksDB's per-table partitioning, with independent in-memory write buffers (memtables) and SSTables) store all information in the key and use the value as a placeholder: in Facebook's social-graph workload (UDB), the `Assoc_count` CF has 20-byte bigint keys with values under 8 B, averaging ∼ 28 B/entry, and similar patterns appear in other index/counter CFs [Cao et al., 2020]. KV-separation deployments (WiscKey [Lu et al., 2016], RocksDB BlobDB, Pebble) store only a key plus a 10–20-byte blob handle in the SSTable, with values placed in separate blob files. In both cases the per-SSTable count rises to $n \in [2^{22},\ 2^{23}]$.

Range filter papers [Zhang et al., 2018, Luo et al., 2020, Vaidya et al., 2022, Eslami Beni et al., 2024, Costa et al., 2024] report headline numbers at $n \in [2^{24},\ 2^{28}]$, but this is a single-filter microbenchmark convention rather than a deployment claim: SuRF, SNARF, and Rosetta integrate per-SSTable into RocksDB; Memento targets B-Trees (WiredTiger, MongoDB's storage engine); Grafite is presented as a stand-alone structure. The SOSD benchmark [Kipf et al., 2019] provides datasets at this same scale (200M keys $\approx 2^{27.6}$).

Why not a global filter. A database-wide filter is faster on the query side: range filter query time is essentially independent of n, so each per-file probe is constant-time — but a per-file design pays one such probe per SSTable, while a single global filter needs only one. The update side is the opposite story: LSM compaction atomically replaces SSTables, making per-file filter rebuild touch only the keys in that compaction, while a global filter must either rebuild over all keys or maintain counting structures that support deletion. GRF [Wang et al., 2024], a recent global range filter for LSM-trees, achieves single-probe queries through shape encoding; production systems — and our benchmarks — still adopt the simpler per-SSTable model.

The query-side argument is also weaker than it first looks. Within a level (L1+), RocksDB stores SSTables as a non-overlapping *sorted run*: each file carries [*smallest*, *largest*] key metadata in the MANIFEST (RocksDB's global level-metadata file), and a query does an $O(\log \#files)$ binary search on this metadata to identify only the SSTables whose key range overlaps the query, then probes their per-file filters [Facebook, 2024b].

Our choice of n. We benchmark at three anchor scales:

- $n = 2^{20}$ — per-SSTable production workload;
- $n = 2^{24}$ — large SSTable or aggregate filter;
- $n = 2^{28}$ — paper-evaluation and SOSD scale.

$n = 2^{30}$ is additionally reported as an asymptotic stretch point, beyond realistic per-SSTable deployments.

Per-key budget. Published range filter evaluations consistently report total per-key budgets in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1). The LSM point-filter convention sits at the lower end of this window: RocksDB ships Bloom filters at 10 BPK by default [Facebook, 2024a], reported by Dayan et al. [2017] as the cross-system standard. We adopt the same target range.

Filter	BPK budget	Reported FPR	Notes
SuRF [Zhang et al., 2018]	10–16	$\sim 10^{-3}$ at 16 BPK*	RocksDB-anchored
Rosetta [Luo et al., 2020]	10–20	10^{-2} to 10^{-4}	FPR-vs-BPK sweep
SNARF [Vaidya et al., 2022]	16	6.2×10^{-5}*	Headline result
Memento [Eslami Beni et al., 2024]	≥ 12	ε-parameterised	Design minimum
Grafite [Costa et al., 2024]	~ 12	$\leq \mathcal{L}/2^{\mathrm{BPK}-2}$	Adversarial bound

Table 6.1: Per-key memory budgets and corresponding false positive rates reported in recent range filter literature. FPR depends strongly on range length $\mathcal{L}$ and workload. *SuRF and SNARF FPR values at 16 BPK are both reported in Vaidya et al. [2022] for direct comparison.

FPR targets. We report, for each filter, the BPK required to reach two fixed FPR targets: 10^{-2} and 10^{-3}. Both targets sit inside the operating range of published range filter evaluations: classical filters [Zhang et al., 2018, Luo et al., 2020, Wang et al., 2024] anchor their headline numbers around 10^{-2} to 10^{-3}, while modern optimal filters [Vaidya et al., 2022, Costa et al., 2024, Eslami Beni et al., 2024] sweep deeper. They also align with the LSM point-filter convention: Cassandra's default `bloom_filter_fp_chance` is 0.01 for all compaction strategies except Leveled Compaction Strategy (LCS) [Apache Software Foundation, 2024].

Query range length $\mathcal{L}$. We sweep $\mathcal{L} \in \{1, 16, 128, 1024, 4096, 16384, 65536\}$. The lower half is production-aligned: in Facebook's UDB workload, more than 60% of iterator operations read exactly one key, the P90 is below 100 keys, and scans above 10,000 are very rare [Cao et al., 2020]; the Yahoo! Cloud Serving Benchmark (YCSB) Workload E caps short-range scans at 100. Range filter papers [Vaidya et al., 2022, Luo et al., 2020, Costa et al., 2024, Eslami Beni et al., 2024] report headline FPR in this same window: SNARF at $\mathcal{L} = 256$, Rosetta at $\mathcal{L} = 64$, Grafite and Memento sweeping $\mathcal{L} \in \{1, 32, 1024\}$. The upper half ($\mathcal{L} \geq 4096$) extends past

production P99 and any reported literature sweep. Such large $\mathcal{L}$ values are usually excluded from filter benchmarks because hashing-based filters cannot maintain low FPR there; we include them because our construction can.

6.3 ERE Backend Comparison: Two-Vector vs One-Vector

We compare the two-vector and one-vector ERE backends as the exact layer of two ARE wrappers (SODA and Greedy+Merge) across six distributions. The analytic prediction from Section 2.1.4 is $\Delta = M/n$ BPK, where M is the number of non-empty blocks; the maximum is $B/n = 1$ BPK and the Poisson middle for uniform input at $\lambda = 1$ is $1 - e^{-1} \approx 0.632$ BPK. The numbers below confirm the prediction end-to-end.

	SODA			Greedy+Merge		
Dataset	two-vec BPK	one-vec BPK	Δ	two-vec BPK	one-vec BPK	Δ
`uniform`	16.932	16.499	0.433	16.932	16.499	0.433
`clustered`	16.602	16.500	0.102	12.759	12.519	0.240
`sosd_fb`	16.020	16.000	0.020	11.232	11.000	0.232
`sosd_wiki`	16.532	16.530	0.002	9.708	9.530	0.178
`sosd_osm`	16.931	16.499	0.432	26.678	26.411	0.267
`sosd_books`	16.000	16.000	0.000	4.517	4.000	0.517

Table 6.2: Total ARE-level BPK with the two-vector (D_1, D_2) ERE backend (Section 2.1.2) versus the one-vector D backend (Section 2.1.4). Workload: $n = 2^{20}$ (or all available keys, whichever is smaller), $K = 34$ — equivalent to $\mathcal{L} = 128$, $\varepsilon = 0.01$ via $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ for SODA; Greedy+Merge takes K directly. SODA stays in the 14–16 BPK headline window (Table 6.1) on every distribution; Greedy+Merge ranges from 4.0 BPK (`sosd_books`, single-cluster collapse) to 26.4 BPK (`sosd_osm`, many small clusters with per-cluster bookkeeping overhead).

SODA's locality-preserving hash keeps clustered inputs clustered in the hashed universe, so the SOSD datasets leave most blocks empty under SODA ($\Delta \leq 0.020$ BPK on `sosd_fb`, `sosd_wiki`, `sosd_books`); `uniform` and `sosd_osm` approach the analytic prediction (Section 2.1.5) at ∼0.43 BPK (because the benchmark evaluates at exactly $n = 2^{20}$, SODA hash collisions drop the unique fingerprint count slightly below the power-of-two threshold; this halves the ERE block count B to 2^{19}, shifting the load factor to $\lambda \approx 2$; the reported ARE BPK is logical, so it perfectly matches the analytic M/n prediction without physical succinct-index overheads). Greedy+Merge isolates dense regions and builds a separate ERE per cluster; even within a cluster the keys retain sub-structure, so M/B stays well below 1. Its absolute BPK is dominated by the wrapper rather than ERE, so even small M/n values translate into a noticeable relative improvement (e.g. `sosd_books` drops from 4.517 to 4.000 BPK).

Query latency. Table 6.3 reports average ARE query latency on the same workload. Both wrappers gain uniformly across all distributions: SODA 1.21–2.95×, Greedy+Merge 1.28–3.00×. The ≥ 2× wins on `uniform` and `sosd_osm` come from cache-friendly access at small suffix width ($w = K - \log_2 n = 14$): the contiguous D vector keeps both SELECT calls hitting the same auxiliary-index pages, while the two-vector layout alternates between D_1 and D_2 and pays an extra cache miss per query. TODO: measure cache hits & misses

	SODA			Greedy+Merge		
Dataset	two-vec ns	one-vec ns	speedup	two-vec ns	one-vec ns	speedup
`uniform`	220.5	75.0	2.94×	218.6	72.9	3.00×
`clustered`	253.9	191.8	1.32×	317.9	198.3	1.60×
`sosd_fb`	309.2	206.7	1.50×	224.9	158.0	1.42×
`sosd_wiki`	266.7	207.2	1.29×	303.3	206.5	1.47×
`sosd_osm`	242.2	82.2	2.95×	298.3	232.6	1.28×
`sosd_books`	263.3	218.3	1.21×	223.1	104.7	2.13×

Table 6.3: Average ARE query latency, two-vector vs one-vector ERE backend. Same workload as Table 6.2: $n = 2^{20}$ (or all available keys, whichever is smaller), $K = 34$; raw 64-bit keys (no 60-bit mask). Apple M4 Max, 32,768 half-hit/half-miss queries, 3 rounds. This is a small-scale backend comparison at fixed $n = 2^{20}$; absolute latencies grow 5–10× at the headline scale $n = 2^{24}$ once the filter footprint outgrows the L3 cache and per-query work becomes cache-miss-bound.

6.4 ERE Bucket Occupancy on SOSD

To characterize how the SODA hash distributes input keys across ERE blocks on real workloads, we measured per-bucket statistics for each SOSD dataset across three range lengths.

Dataset	n	$\mathcal{L}$	w	X_{50}	X_{90}	Max	Max footprint	Fill
`sosd_fb`	2^{27}	16	11	27	103	463	637 B	22.6%
`sosd_fb`	2^{27}	256	15	208	545	1,589	2.9 KB	4.9%
`sosd_wiki`	$\sim 2^{26*}$	16	12	3,240	3,843	4,054	5.9 KB	**99.0**%
`sosd_wiki`	$\sim 2^{26*}$	256	16	52,565	58,855	62,037	121 KB	**94.7**%
`sosd_books`	2^{27}	16	11	812	875	978	1.3 KB	47.8%
`sosd_books`	2^{27}	256	15	13,215	13,670	14,039	25.7 KB	42.8%
`sosd_osm`	2^{29}	16	11	3	5	35	48 B	1.7%
`sosd_osm`	2^{29}	256	15	3	5	85	159 B	0.3%

Table 6.4: Bucket-size statistics for the inner ERE of the SODA-hashed ARE filter on the SOSD datasets across two range lengths covering the typical Facebook UDB range ($\mathcal{L} = 16$) and its P90 ($\mathcal{L} = 256$). *`sosd_wiki` effective $n \approx 65 \times 10^6$ after deduplication of repeated timestamps (Section 3.1.2); reported as $\sim 2^{26}$ for visual consistency with the other rows.

Columns.

w — Suffix length in bits, configured per row. Physical block capacity is 2^w distinct w-bit suffixes.

X_{50}, X_{90} — Key-weighted CDF percentiles over non-empty buckets: the smallest bucket size such that buckets of size $\leq X_p$ collectively hold at least $p \cdot n$ keys.

Max — Empirical maximum bucket size, $\max_b k_b$.

Max footprint — Bit-packed size of the heaviest bucket ($\max_b k_b \cdot w/8$ bytes).

Fill — $\max_b k_b / 2^w$ — the heaviest bucket's saturation against physical capacity.

Observations. `sosd_fb` and `sosd_osm` stay well below saturation at every $\mathcal{L}$. `sosd_books` runs at ∼42% across $\mathcal{L}$. `sosd_wiki` fully saturates at $\mathcal{L} = 16$ (99%). In all cases except `sosd_wiki` at $\mathcal{L} = 256$, bucket footprints fit L1d on modern x86 servers, typically 32–48 KB. The saturated wiki bucket (121 KB) fits L2.

When fill approaches 100%, the SODA hash has run out of resolution: every key in a dense input region collapses into a single ARE block, and the bucket-search cost for those queries becomes $O(2^w)$ rather than $O(1)$ on average. For `sosd_wiki` this means binary search on a fully packed bucket of up to $\sim 6 \times 10^4$ keys (121 KB at $\mathcal{L} = 256$) — still bounded by w iterations and cache-resident in L2 (Section 2.1.5), but the buckets are no longer lightly loaded as the SODA construction assumes.

6.5 Large-Range Performance: $\mathcal{L} = 65536$

The combination of truncation-based hashing and exact-mode clusters enables handling very large range queries that would be impractical for other approaches.

At $\mathcal{L} = 65536$, non-hybrid filters face a fundamental problem: the fingerprint length $K = \lceil \log_2(n \cdot \mathcal{L}/\varepsilon) \rceil$ must grow by $\log_2(\mathcal{L}) = 16$ bits to maintain the same FPR. For SODA, this means 16 extra BPK. For Bloom, $\mathcal{L} = 65536$ point lookups per query is impractical.

Scan-ARE sidesteps this on two levels:

Exact clusters. Each cluster stores its $[\min, \max]$ bounds. If a query range $[a, b]$ overlaps a cluster's bounds, at least one stored key is inside $[a, b]$ — that is a true positive, not a false positive. If $[a, b]$ does not overlap any cluster, the cluster contributes nothing. If $[a, b]$ falls entirely within a cluster, the sub-filter is exact (Section 4.3), so false positives are impossible. Either way, exact clusters produce zero false positives regardless of $\mathcal{L}$.

Truncation fallback. The sparse fallback uses truncation hashing (Section 4.2), where the phantom points of a stored key are confined to the 2^t-point prefix block containing that key (Figure 4.3). A false positive can occur only when a query *endpoint* a or b falls inside such a block without containing the stored key itself. Because these phantoms remain localized rather than scattering across the universe as under SODA, the FPR of truncation is independent of the query range length $\mathcal{L}$. The fallback thus avoids the $\log_2(\mathcal{L})$ BPK penalty that SODA-based filters must pay.

6.5.1 SOSD Results ($n = 2^{24}$, $\mathcal{L} = 65536$)

On both datasets, Scan-ARE achieves FPR $< 10^{-8}$ at ∼5 BPK, while Truncation, SODA, and Bloom all degrade to FPR ≈ 1 at comparable budgets. SNARF plateaus at FPR $\sim 10^{-3}$. Only Grafite remains competitive on Wiki (∼10 BPK for FPR 10^{-2}), but at 2–3× the space of Scan-ARE.

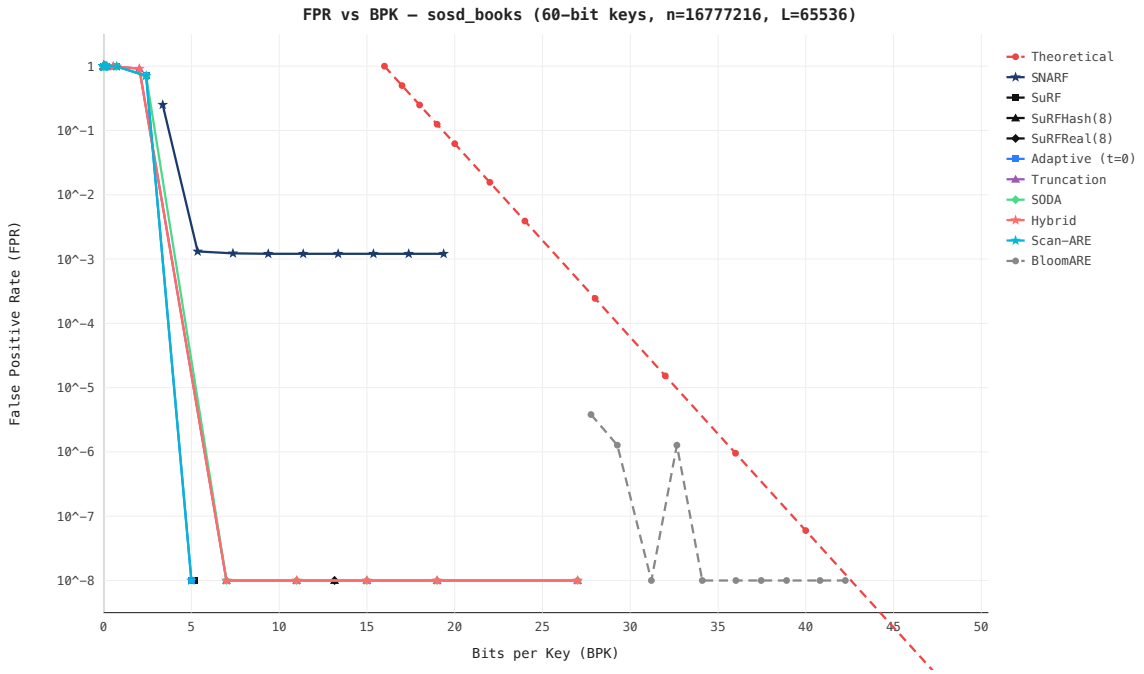

Figure 6.1: FPR vs BPK — SOSD Books (ISBN sequences, highly clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE achieves FPR $< 10^{-8}$ at $\sim$5 BPK; all non-hybrid filters degrade to FPR ≈ 1.

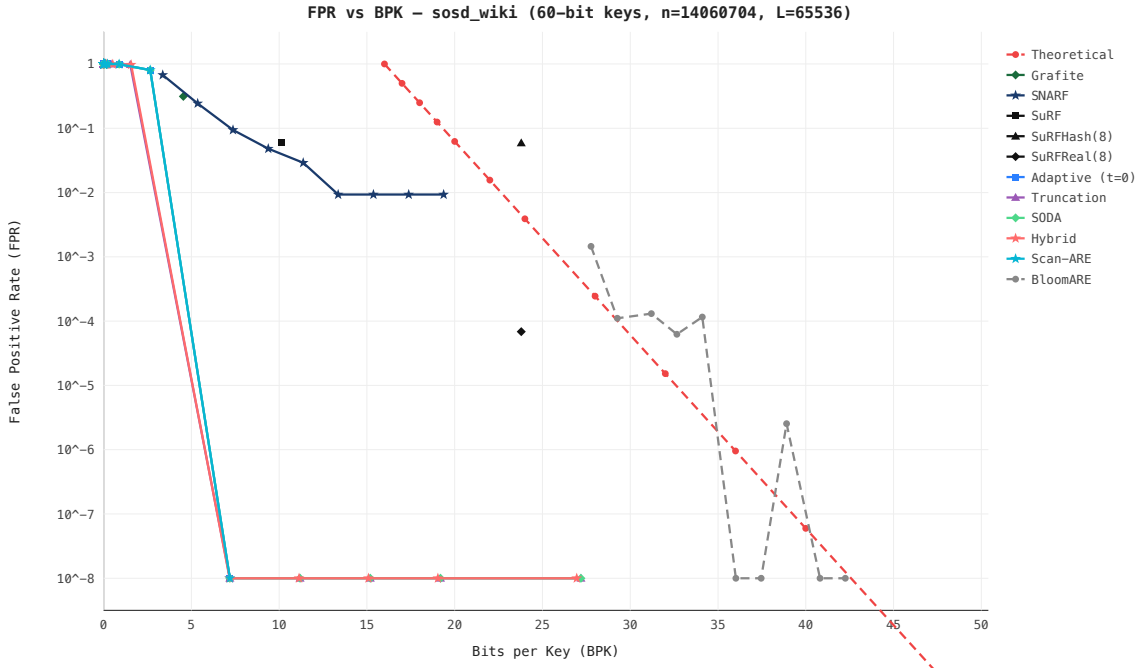

Figure 6.2: FPR vs BPK — SOSD Wiki (Wikipedia timestamps, moderately clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE dominates; only Grafite remains competitive at $\sim$10 BPK but with 2–3$\times$ more space.

6.6 BPK at Industry-Standard FPR Targets

In practice, storage systems target a specific FPR — typically 10^{-2} or 10^{-3} — and want to minimize the filter's memory footprint. The relevant question is: how many bits per key does each filter need to reach the target FPR?

We report this metric separately for each of the 9 benchmark distributions: 4 real-world (SOSD) and 5 synthetic. Each distribution exercises different filter strengths — a filter that excels on clustered data may be mediocre on uniform, and vice versa.

SOSD Distributions

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.5: BPK to reach target FPR — SOSD Facebook ($n = 2^{24}$, highly clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.6: BPK to reach target FPR — SOSD Books ($n = 2^{24}$, clustered by publisher).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.7: BPK to reach target FPR — SOSD Wiki ($n = 2^{24}$, moderately clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.8: BPK to reach target FPR — SOSD OSM ($n = 2^{24}$, geospatially clustered).

Synthetic Distributions

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.9: BPK to reach target FPR — Clustered synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.10: BPK to reach target FPR — Uniform synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.11: BPK to reach target FPR — Spread synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.12: BPK to reach target FPR — Zipfian synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.13: BPK to reach target FPR — Temporal synthetic ($n = 2^{24}$).

6.7 Summary of Results

Filter	BPK ($\mathcal{L}$=65536)	FPR	Distribution dependence
Scan-ARE	∼5	$< 10^{-8}$	Best on clustered data
Grafite	∼10	$\sim 10^{-2}$	Distribution-independent
SNARF	∼28	$\sim 10^{-3}$	Learned, data-dependent
SODA	∼5	≈ 1 (degenerate)	Distribution-independent
Truncation	∼3	≈ 1 (degenerate)	Fails on clustered data
Bloom	∼30	$\sim 10^{-4}$	Distribution-independent

Table 6.14: Comparison at $n = 2^{24}$, $\mathcal{L} = 65536$ on SOSD Books. Scan-ARE dominates the Pareto frontier by exploiting cluster density for exact-mode sub-filters.

Chapter 7

Conclusion

Bibliography

Apache Software Foundation. Apache Cassandra documentation: Bloom filters, 2024. URL `https://cassandra.apache.org/doc/latest/cassandra/managing/operating/bloom_filters.html`. Accessed: 2026-04-26.

Djamal Belazzougui, Paolo Boldi, Rasmus Pagh, and Sebastiano Vigna. Monotone minimal perfect hashing: Searching a sorted table with O(1) accesses. In *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '09)*, pages 785–794. SIAM, 2009.

Djamal Belazzougui, Paolo Boldi, Rasmus Pagh, and Sebastiano Vigna. Fast prefix search in little space, with applications. In *ESA (1)*, volume 6346 of *LNCS*, pages 427–438. Springer, 2010.

Zhichao Cao, Siying Dong, Sagar Vemuri, and David H. C. Du. Characterizing, modeling, and benchmarking RocksDB key-value workloads at Facebook. In *18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 209–223. USENIX Association, 2020.

Marco Costa, Paolo Ferragina, and Giorgio Vinciguerra. Grafite: Taming adversarial queries with optimal range filters. In *Proceedings of the 2024 International Conference on Management of Data (SIGMOD '24)*. ACM, 2024. doi: 10.1145/3639258.

Niv Dayan, Manos Athanassoulis, and Stratos Idreos. Monkey: Optimal navigable key-value store. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*, pages 79–94. ACM, 2017. doi: 10.1145/3035918.3064054.

Martin Dietzfelbinger. Universal hashing and k-wise independent random variables via integer arithmetic without primes. In *Proceedings of the 13th Annual Symposium on Theoretical Aspects of Computer Science (STACS '96)*, volume 1046 of *LNCS*, pages 569–580. Springer, 1996.

Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. Evolution of development priorities in key-value stores serving large-scale applications: The RocksDB experience. In *19th USENIX Conference on File and Storage Technologies (FAST '21)*, pages 33–49. USENIX Association, 2021.

Dominik Durner, Viktor Leis, and Thomas Neumann. Exploiting cloud object storage for high-performance analytics. *Proceedings of the VLDB Endowment*, 16(11):2769–2782, 2023. doi: 10.14778/3611479.3611486.

Peter Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM*, 21(2):246–260, 1974.

Navid Eslami Beni, Ioana O. Bercea, and Niv Dayan. Memento filter: A fast, dynamic, and robust range filter. In *Proceedings of the 2024 International Conference on Management of Data (SIGMOD '24)*. ACM, 2024. doi: 10.1145/3698820.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96)*, pages 226–231, 1996.

Facebook. RocksDB Bloom filter, 2024a. URL `https://github.com/facebook/rocksdb/wiki/RocksDB-Bloom-Filter`. Accessed: 2026-04-26.

Facebook. RocksDB leveled compaction, 2024b. URL `https://github.com/facebook/rocksdb/wiki/Leveled-Compaction`. Accessed: 2026-04-26.

Robert M Fano. On the number of bits sufficient to implement an associative memory. In *Memorandum 61, Project MAC, MIT*, 1971.

William Feller. *An Introduction to Probability Theory and Its Applications, Vol. 1.* Wiley, 3 edition, 1968.

Google. S2 Geometry Library, 2017. URL `https://s2geometry.io`. Accessed: 2026-03-25.

Mayank Goswami, Allan Grønlund Jørgensen, Kasper Green Larsen, and Rasmus Pagh. Approximate range emptiness in constant time and optimal space. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '15)*, pages 848–858. SIAM, 2015.

Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. SOSD: A benchmark for learned indexes. *NeurIPS Workshop on Machine Learning for Systems*, 2019. URL `https://arxiv.org/abs/1911.13014`.

Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. WiscKey: Separating keys from values in SSD-conscious storage. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*, pages 133–148. USENIX Association, 2016.

Siqiang Luo, Subhadeep Chatterjee, Rafael Ketsetsidis, Niv Dayan, Wilson Qin, and Stratos Idreos. Rosetta: A robust space-time optimized range filter for key-value stores. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, pages 2071–2086. ACM, 2020.

Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Structures*. Cambridge University Press, 2 edition, 2017.

Kapil Vaidya, Subhadeep Chatterjee, Eric Knorr, Michael Mitzenmacher, Stratos Idreos, and Tim Kraska. SNARF: A learning-enhanced range filter. *Proceedings of the VLDB Endowment*, 15(8):1632–1644, 2022.

Hengrui Wang, Te Guo, Junzhao Yang, and Huanchen Zhang. GRF: A global range filter for LSM-trees with shape encoding. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654944.

Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*, pages 323–336. ACM, 2018.